

2025 年春季学期

数据结构课程设计赛道 B 实验报告

马兆星¹ 郭怡豪²

¹ 计算机科学与技术学院 2023 级 4 班

² 计算机科学与技术学院 2023 级 4 班

1 分工与合作

◆ 马兆星(50%):

棋盘(AtaxxBoard)位编码数据结构设计与实现
高效位运算操作的对应系列函数开发
MCTS 算法的选择(selectBestChild)和回溯(updateNodeValue)策略实现
MCTS 算法性能调优与时间控制机制实现
撰写实验报告和 botzone 平台测试分析

◆ 郭怡豪(50%):

MCTS 节点(MCTSNode)数据结构设计与实现
合法移动生成函数(generateLegalMoves)开发
MCTS 算法的扩展(expandChildren)
终局判断(checkGameOver)策略实现

2 算法思想

2.1 总体思路

总体采用蒙特卡洛树搜索(MCTS)算法作为同化棋 AI 的核心决策机制。

◆ 数据结构设计:

1. 棋盘(AtaxxBoard)采用位编码[1]表示:

boardState[7] 数组存储 7×7 棋盘。其中每个位置用 2 位表示 (0x=空, 10=黑, 11=白, 即前一位可判断是否存在棋子, 后一位可判断棋子具体颜色)。
通过位运算而非循环嵌套高效访问, 使得遍历棋盘及同化操作高效。[例如
 $\text{boardState}[x] \gg ((y+1)*2) \& 3$
playerColor 则用于记录当前行动方颜色。

2. MCTS 节点(MCTSNode):

boardState[7]存储棋盘状态，playerColor 记录当前玩家
 move_from_parent 存储从父节点到当前节点的移动操作
 children 指针数组存储所有子节点，child_count 记录子节点数量
 visit_count 记录节点访问次数，total_score 累计所有模拟得分
 terminal_score 缓存终局评分
 支持高效的节点选择、扩展、评估和回溯操作，同时采用动态分配，仅在需要时分配子节点内存，节约资源结构，使用 UCB 公式平衡探索与利用。

◆ MCTS 算法策略[2]:

1. 选择(selectBestChild):

使用 UCB 公式计算每个子节点价值，平衡已知的好节点，探索未充分探索的节点，时间内选择最有潜力的节点。

公式核心：平均得分 - 探索系数 * $\sqrt{\ln(\text{父节点访问次数}) / \text{当前节点访问次数}}$

2. 扩展(expandChildren):

利用 generateLegalMoves 函数生成所有合法移动，然后为每个合法移动创建新的子节点，通过 makeMove 应用移动，更新相应的状态。

3. 模拟(evaluatePosition):

不执行传统的完全随机模拟，而是直接使用启发式评估函数。评估函数计算当前玩家与对手棋子数量差，作为当前局面价值。终局情况使用 checkGameOver 判断，返回固定的终局分数 ($\pm \text{TERMINAL_SCORE}$)。

4. 回溯(updateNodeValue):

更新节点的累计得分 total_score

因为是**零和游戏**，子节点向父节点传递的分数需要取反

算法在 1s 时间内执行尽可能多的 MCTS 迭代，选择访问次数最多的移动作为最佳移动。算法兼顾了搜索效率和决策质量，适合具有较大状态空间的同化棋游戏，在 botzone.org.cn 上天梯实测表现尚佳。

2.2 所用方法的特别、新颖或创新之处

在标准 MCTS 算法基础上融入了几项优化技术，提升 AI 在同化棋游戏中的性能

1. **位操作[1]表示**：7x7 棋盘仅用 7 个 int 存储节约了大量内存，又通过 $(y+1) \ll 1$ 的位偏移计算实现了快速的随机访问，同时预计算的 piece_conversion_mask 实现单指令多棋子同化。相比传统二维数组不仅显著提高了同化棋 AI 的操作速度，又实现了更高效的内存使用和快速状态更新。

2. **UCB 探索策略改进**: 对未访问节点采用强制探索机制 ($-\text{MAX_SCORE_VALUE} + \text{随机扰动}$), 确保搜索空间全覆盖的同时, 通过随机因子有效解决对称局面的决策困境, 确保了所有节点都有机会被探索, 同时通过随机因子打破平局, 显著增加了搜索多样性。

3 总结

在完成本项目过程中, 我组遇到了多项技术挑战, 其中最为突出的问题主要集中在位操作的调试困难和搜索算法的效率瓶颈。尤其在 **botzone** 的 **1s** 时间限制下, 初期实现的搜索算法表现很差, 尤其是当对手采取复杂策略时, 算法响应缓慢甚至超时。通过分析平台战局回放数据, 发现主要瓶颈源于冗余计算和评估函数设计不合理。经过多轮优化和引入迭代加深和时间控制机制, **AI** 在比赛中的表现明显提升。

虽然使用 **MCTS** 算法, 但缺乏针对同化棋特性的专门剪枝策略, 在某些特殊局面下仍会浪费计算资源在低价值分支上, 同时还有想实现但未能实现有效的开局库, 导致在游戏早期阶段决策质量不够稳定, 综上该代码仍有诸多可提升空间。

通过这次实践, 我组深刻体会了理论与工程实现之间的差距, 以及在受限环境下优化的复杂性。这锻炼了我们的问题分析和解决问题的能力, 更让我们认识到在 **AI** 博弈系统中, 性能与策略间的平衡艺术才是成功的关键。这些宝贵经验将指导我们在未来的算法设计中更加注重实际约束条件, 而非仅追求理论完美。

4 参考文献

[1] 数据结构——位棋盘 (2015-07-15)

https://www.xqbase.com/computer/struct_bitboard.htm

[2] 蒙特卡洛树搜索 (MCTS) 在大模型推理中的作用与应用 (2025-02-25)

https://blog.csdn.net/shizheng_Li/article/details/145860039